

Laboratuvardan Vitrine: Gradio ile Makine Öğrenmesi Modellerini Web Arayüzlerine Dönüştürmek

Hugging Face Ekosisteminde Etkileşimli Araçlar Geliştirme ve Paylaşma

Bilişim Sistemleri ve Teknolojileri,
4. Sınıf Akademik Sunumu

Gemini Notebook

Makine Öğrenmesindeki Son Kullanıcı Uçurumu



Gradio'nun Çözümü



Geliştirme Ortamı (Kara Kutu)

API'ler, uçbirimler ve Jupyter defterleri teknik olmayan kullanıcılar için erişilemezdir.

Kullanım Ortamı (Etkileşimli Vitrin)

Modellerin gücünü sergilemek için sezgisel, görsel ve web tabanlı bir arayüze ihtiyaç vardır.

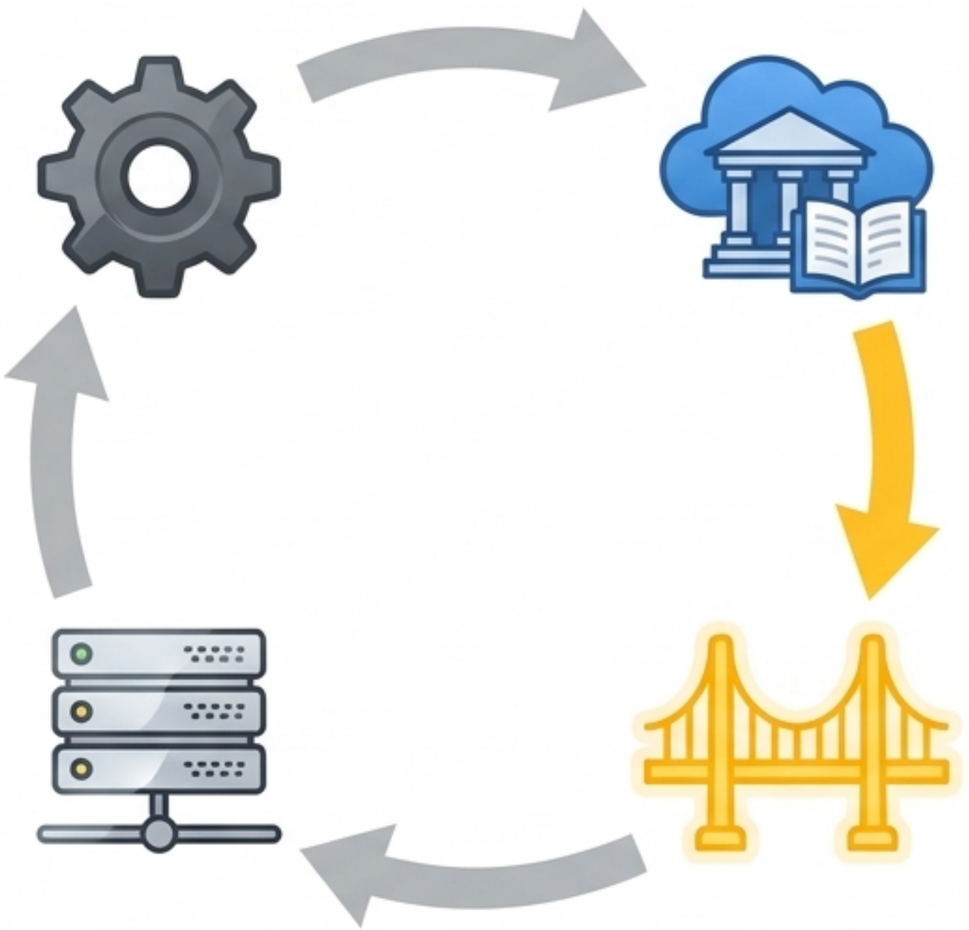
Gemini Notebook

Developer Studio

Hugging Face Ekosistem Haritası

Motor (Transformers & Datasets)

Modellerin eğitildiği ve çalıştırıldığı arka plan.



Depo (Hub)

Açık kaynaklı küresel model kütüphanesi.

Arayüz (Gradio)

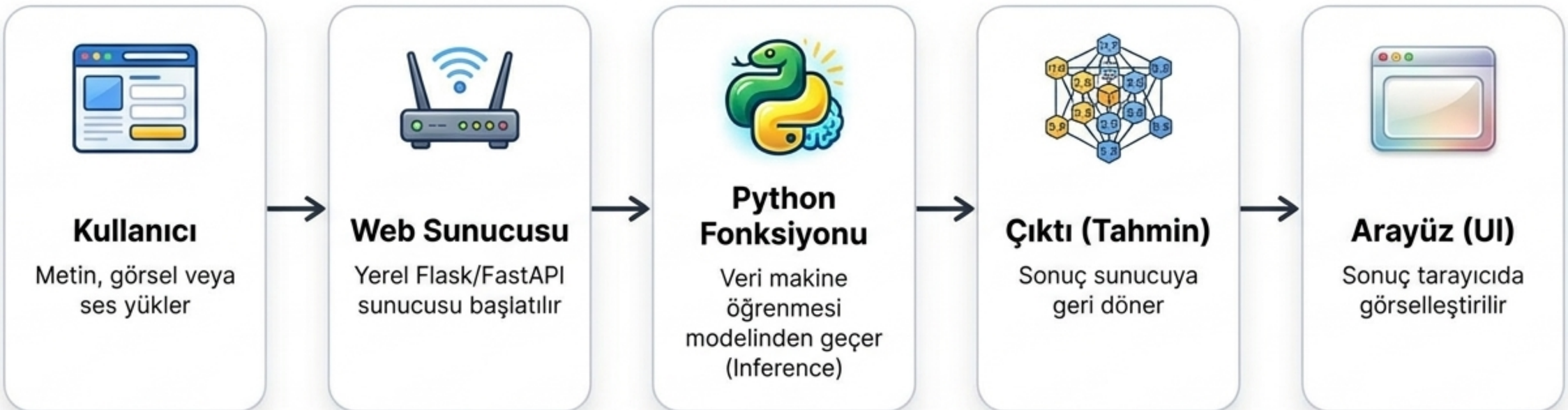
Modeli web bileşenlerine bağlayan iletişim katmanı.

Vitrin (Spaces)

Uygulamanın dünyaya açıldığı kalıcı barındırma alanı.

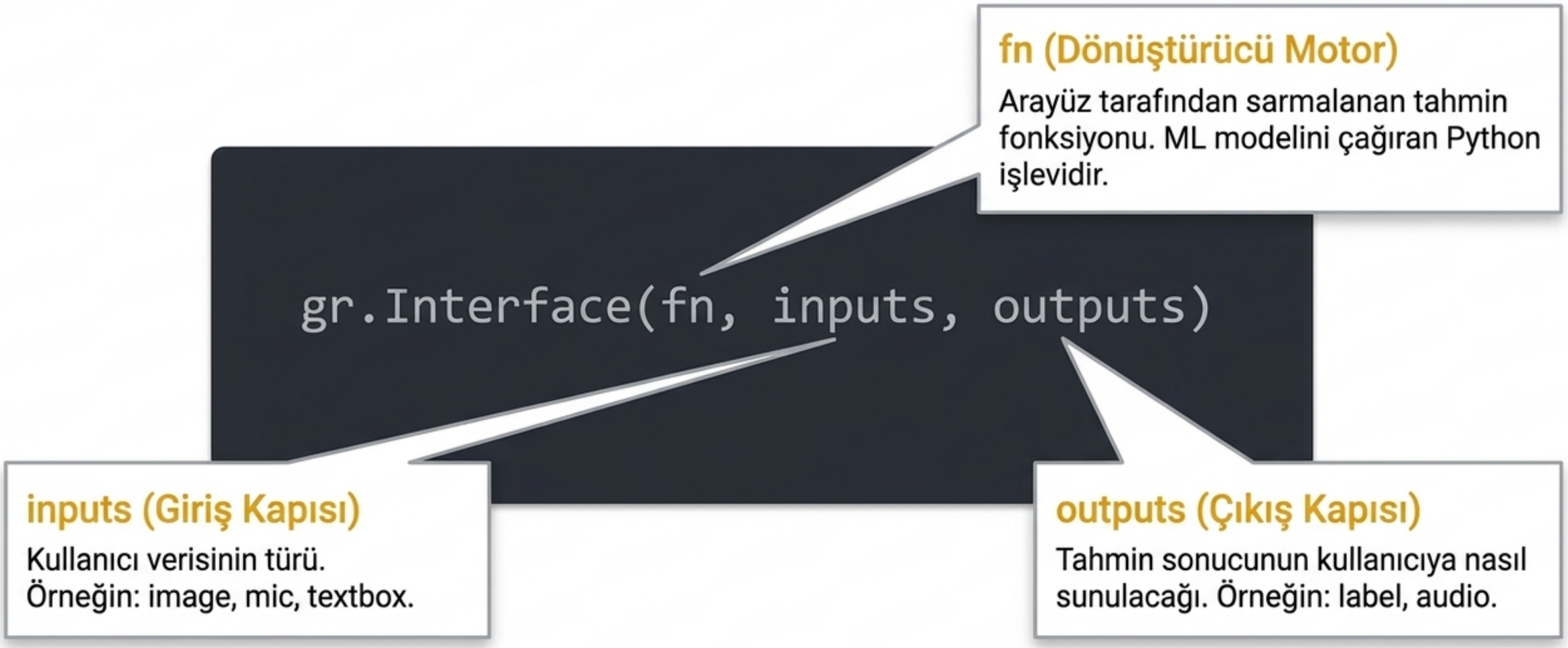
Gemini Notebook

Gradio Mimarisi: Kaputun Altında Ne Var?



Gemini Notebook

Temel API Sınıfı: gr.Interface Anatomisi



Her bileşen özel sınıflarla (ör. gr.Textbox) detaylandırılabilir.

Gemini Notebook

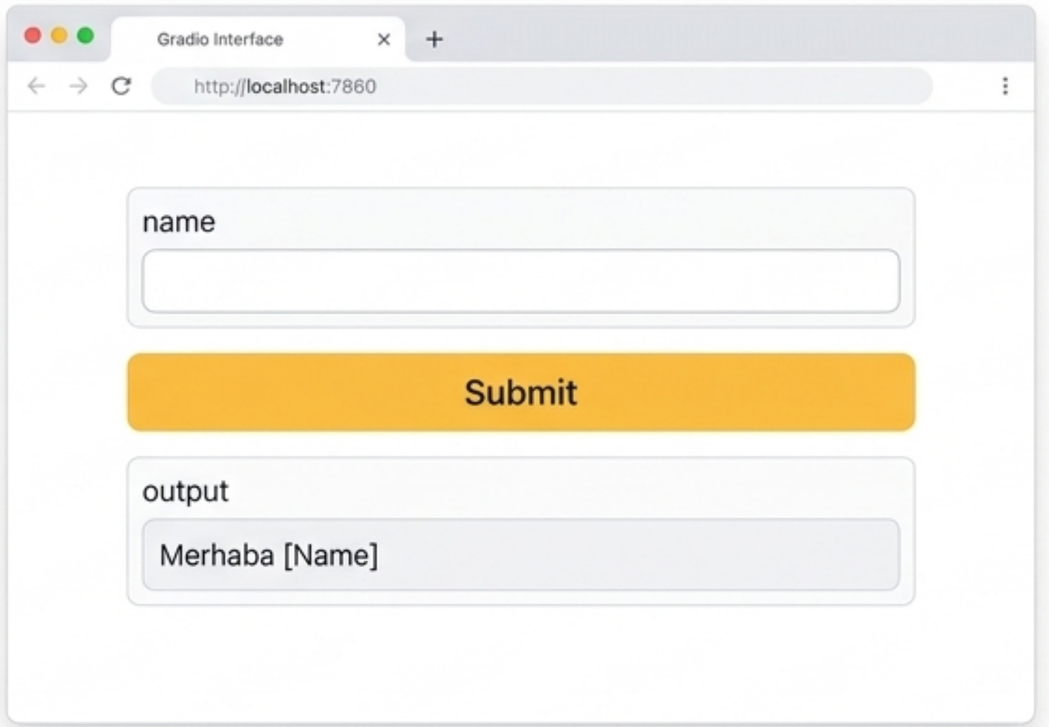
İlk Uygulama: Merhaba Dünya

```python
import gradio as gr

def greet(name):
    return 'Merhaba ' + name

demo = gr.Interface(
    fn=greet,
    inputs='text',
    outputs='text'
)

demo.launch()
```



.launch() metodu, uygulamayı yerel web sunucusunda (http://localhost:7860) başlatır.

Gemini Notebook

Gerçek Model Entegrasyonu: Metin Üretimi (GPT-2)

1. Modelin Yüklenmesi (Transformers)

```
generator = pipeline('text-generation', model='gpt2')
```

2. Sarmalayıcı Fonksiyon (Adaptör)

```
def predict(prompt):
    return generator(prompt)[0]['generated_text']
```

3. Arayüze Bağlama (Gradio)

```
gr.Interface(fn=predict, inputs='text', outputs='text').launch()
```

Gemini Notebook

Çoklu Modalite: Farklı Veri Tipleri

Metin (Text)

```
gr.Textbox()
```

Metin analizi, çeviri ve LLM diyalogları için kullanılır.

Ses (Audio)

```
gr.Audio(source='microphone', type='numpy')
```

Ses girişini otomatik olarak örnekleme hızı ve ses verisinden oluşan bir Numpy dizisi olarak iletir.

Görüntü (Image)

```
gr.Image()
```

Görüntü sınıflandırma veya nesne tespiti modelleri için pikselleri matrise dönüştürür.

Çoklu Parametre (Lists)

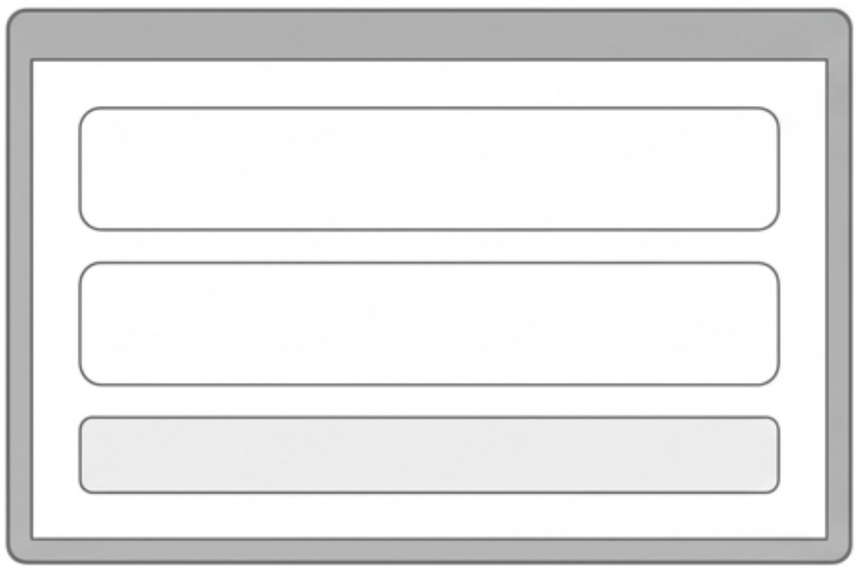
```
inputs=[gr.Slider(), gr.Number()]
```

Fonksiyon birden fazla parametre alıyorsa, bileşenler listeler [] halinde iletilir ve sırayla eşleşir.

Gemini Notebook

Arayüz Tasarımı ve Kullanıcı Deneyimi (UX/UI)

Ham Arayüz



```
title
description
theme
examples
live=True
```

Özelleştirilmiş Arayüz



- **title / description**: Kullanıcıya bağlam sunar.
- **theme**: Görsel estetiği değiştirir.
- **examples**: Tıklanabilir hazır girdi örnekleri sunarak kullanımı kolaylaştırır.
- **live=True**: Kullanıcı girdisi değiştikçe modeli anında çalıştırır.

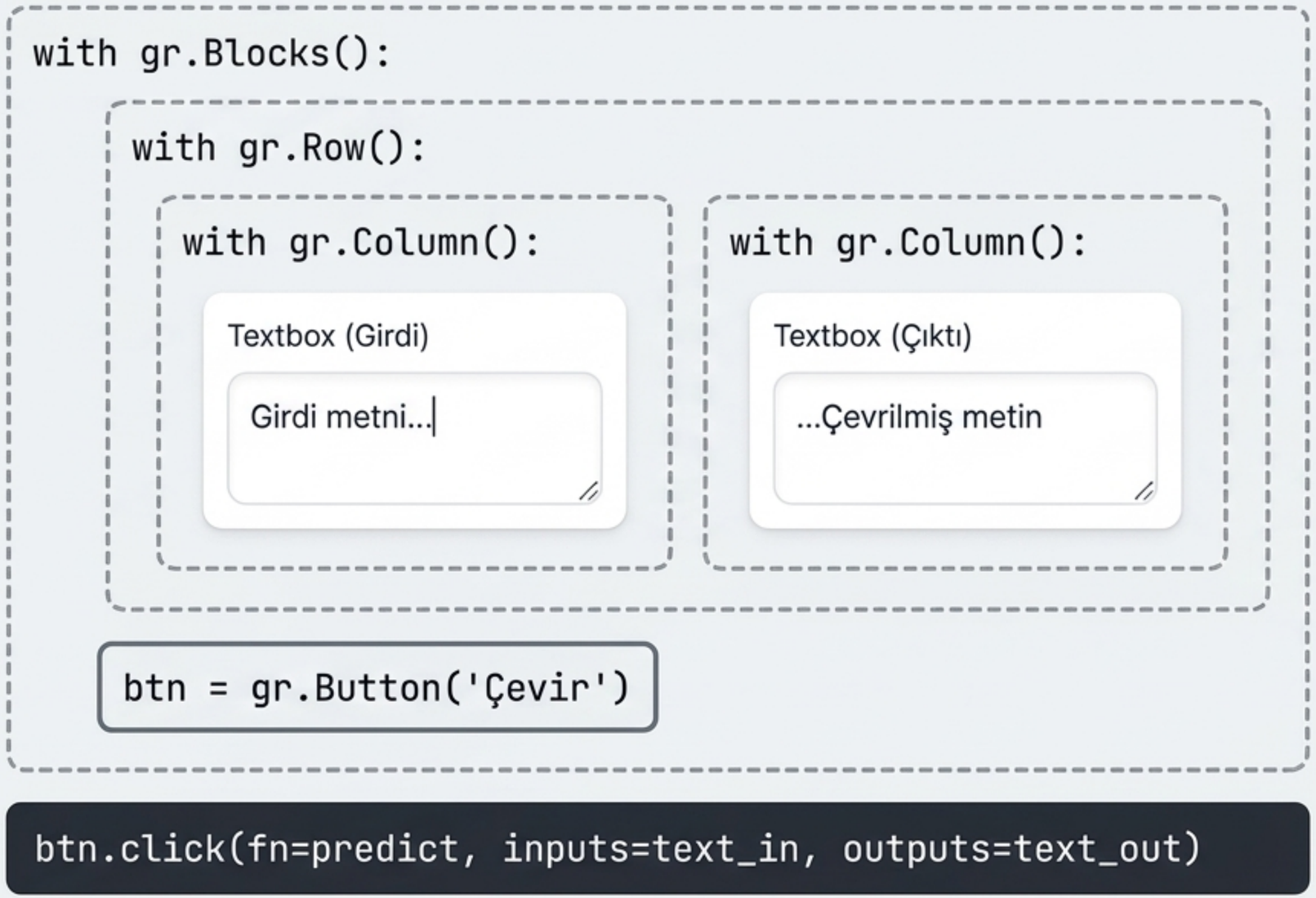
Gemini Notebook

Sentez Karşılaştırması: Interface vs. Blocks

	🪄 gr.Interface	🧱 gr.Blocks
Kullanım Amacı	Hızlı ML demoları ve prototipler oluşturmak.	Uygulamanın veri akışı ve düzeni üzerinde tam kontrol sağlamak.
Esneklik	Düşük. Standart yukarıdır aşağıya akış (Girdi -> Düğme -> Çıktı).	Yüksek. Serbest sayfa düzeni ve sekme yapıları eklenebilir.
Olay Tetikleyicileri	Otomatik. Tek bir Submit butonu ile çalışır.	Manuel. Olay dinleyicileri (btn.click, text.change) spesifik atanır.
API Seviyesi	Yüksek Seviyeli (High-Level) API	Düşük Seviyeli (Low-Level) API

Gemini Notebook

Mimari Kontrol: Gradio Blocks




```
btn.click(fn=predict, inputs=text_in, outputs=text_out)
```

💠 - **Context Manager:** Python 'with' bloğu kullanılarak hiyerarşik UI inşa edilir.

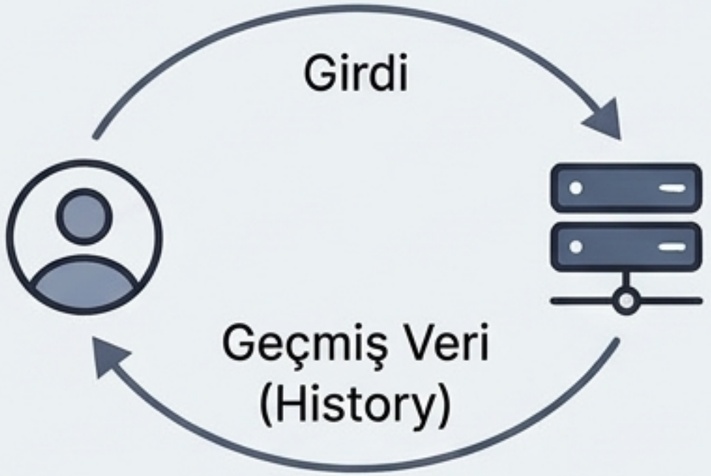
🧱 - **Düzen (Layout):** Bileşenleri dikey (Column) veya yatay (Row) dizer.

🪄 - **Olay Dinleyicileri:** Fonksiyonları spesifik UI etkileşimlerine bağlar.

Gemini Notebook

İleri Düzey Özellikler: Oturum (State) ve Yorumlama

Kalıcı Veri (Session State)



Sayfa yüklemesi süresince kullanıcının geçmiş verilerini saklar (Örn: Chatbot).

Model farklı kullanıcılar arası veri paylaşmaz, sadece aktif oturumu hatırlar.

Model Yorumlama (Explainable AI)

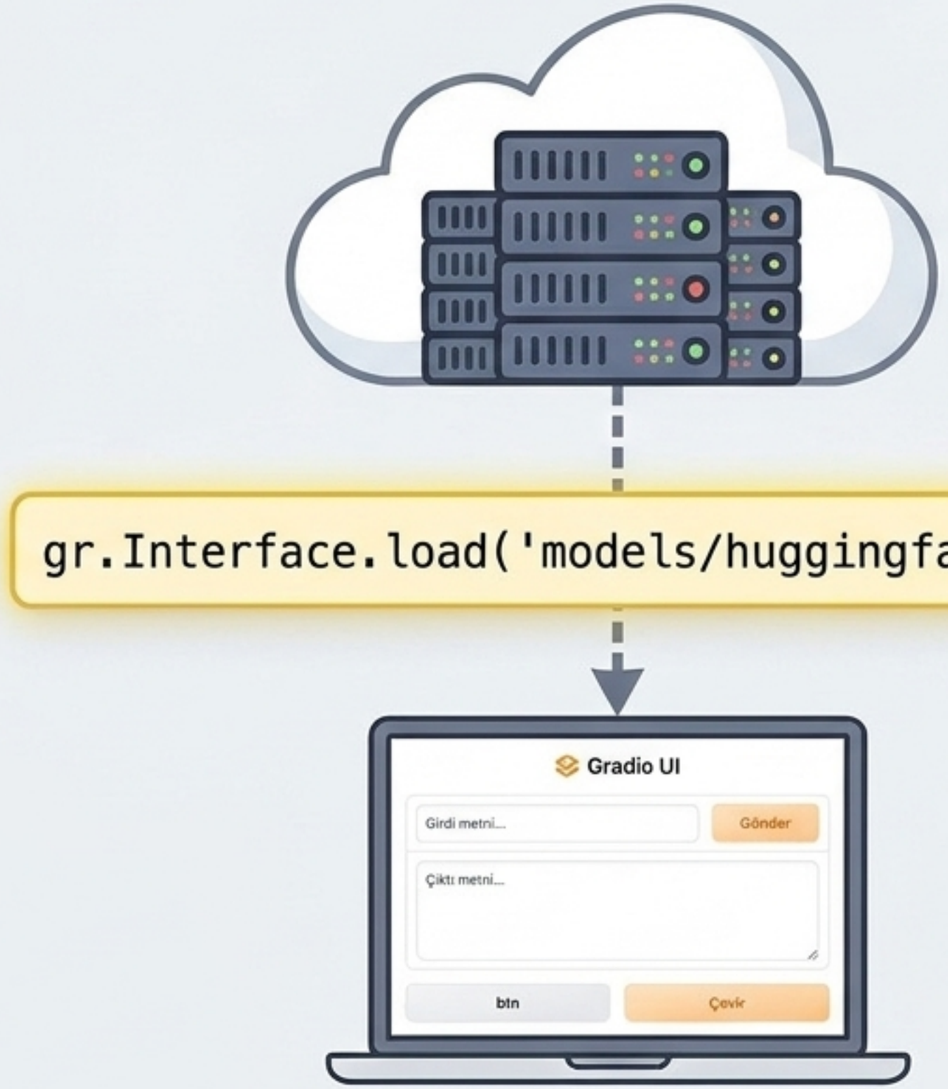


interpretation='default' kullanılarak, girdinin hangi kısımlarının model çıktısından sorumlu olduğu görselleştirilir.

Kara kutu algısını yıkar.

Gemini Notebook

Hugging Face Çıkarım API Entegrasyonu



❌ - **Sorun:** Büyük dil modelleri yerel bilgisayarın RAM'ini doldurur ve indirmesi saatler sürer.

✅ - **Çözüm:** load() komutu ile model yerel belleğe indirilmez.

⚙️ - **Mekanizma:** Tüm çıkarım (Inference) işlemleri Hugging Face'in güçlü bulut sunucularında gerçekleştirilir, sadece sonuçlar arayüzünüze döner.

Gemini Notebook

Demoyu Dünyaya Açmak: Paylaşım ve Barındırma

`Hugging Face Spaces`

Kalıcı Barındırma (Spaces)

Proje dosyalarının bulut deposuna gönderilerek internette 7/24 ücretsiz, kalıcı bir demo olarak sunulmasıdır.

`.launch(share=True)`

Geçici Tünel

Gradio proxy sunucusu üzerinden 72 saatlik, dışarıdan erişilebilir şifreli bir link (XXXX.gradio.app) açar. Cihaz açık kalmalıdır.

`.launch()`

Yerel (Localhost)

Yalnızca geliştiricinin kendi makinesinde çalışır. Dışarı kapalıdır.

⚠ **Uyarı:** share=True kodun sizin bilgisayarınızda çalışmasına neden olur. Hassas veriler gönderilmemeli ve sistem güvenliğine dikkat edilmelidir.

Gemini Notebook

Sonuç, Teknik Değerlendirme ve Q&A

Soru 1

share=True kullanırken veriler Gradio sunucularında saklanır mı?

Hayır, Gradio sadece yerel makinenize bir proxy (tünel) açar. Ancak link public olduğu için gönderilen veriler hassas hassas olmamalıdır.

Soru 2

Oturum (State) değişkeninin Python'daki Global değişkenden farkı nedir?

Global değişkenler tüm kullanıcılar arası ortak hafıza yaratır. State ise sadece sayfayı açan o spesifik kullanıcının oturumunda veriyi tutar.

Soru 3

Neden karmaşık uygulamalarda Interface yerine Blocks tercih edilmeli?

Blocks, model çıktılarını başka bir modelin girdisi olarak bağlamanıza ve DOM üzerinde olay tabanlı kontrol sağlamanıza olanak tanır.

Gradio, geliştirici hızı ile son kullanıcı deneyimi arasındaki mükemmel dengeyi kurar.

Kaynak: Hugging Face NLP Ders Notları (Bölüm 9: NLP'nin Ötesine Geçmek - Gradio Demoları)

Gemini Notebook